AlphaModel diagram

Figure 1: AlphaModel diagram

AlphaModel architecture (applied-science view)

This document describes **AlphaModel** as implemented in this codebase, with shape-level clarity and code pointers. AlphaModel is the forecasting half of the system: it produces a **probabilistic distribution over next-step returns** for a large equity universe, and exposes an expected-return signal `mu` used by downstream portfolio policies.

Canonical implementation (multi-ticker path): - `train/run100_model.py :: Run100FullSeqModel.forward_multi` - `models/price_encoder_with_mlp.py :: PriceEncoderWithMLP` - `models/cross_ticker_attention.py :: CrossTickerAttention` - `train/run100_model.py :: SharedOrdinalHead` - `scripts/run125_alpha_policy.py :: compute_mu` (mu = E[return] from predicted distribution)

Diagram: `docs/alpha_model_diagram.png`

---

1. Problem statement and modeling choice

AlphaModel is trained to produce, for each ticker and each day, a **full discrete probability distribution** over return bins. This design supports: - **Calibration-aware learning** (via distribution losses such as CE/CRPS in the trainer), - Robust ranking and risk-aware post-processing, - A clean interface to portfolio allocation (consume `mu` and uncertainty features if desired).

AlphaModel is intentionally **unified across tickers**: one model predicts for an entire universe, sharing parameters and learning cross-sectional structure via cross-ticker attention.

---

2. Inputs and batch contract (Run101MultiTickerDataset)

AlphaModel is typically driven using the **aligned multi-ticker dataset**:

Entry point (single sample): - `train/run101_multi_dataset.py :: Run101MultiTickerDataset.__getitem__`

It emits a **single sample** containing all tickers aligned to the same max sequence length `L_max` via padding.

2.1 Primary tensors

For a universe of `N` tickers and sequence length `L`:

- `price_seq_stack`: **[N, L, 6]**
  Normalized OHLCV + a return channel (see your run101 builder; in this repo the MLP feature builder uses: open/high/low/close/volume_z + `lr` in the last channel).
- `news_emb_stack`: **[N, L, K, Dn]**
- `news_mask_stack`: **[N, L, K]**
- `news_intensity_stack`: **[N, L, I]**
- `ticker_mask_stack`: **[N, L]** (True = valid token, False = padding)
- `split_mask_stack[split]`: **[N, L]** (True = belongs to split)
- Labels (per ticker per time step):
  - `target_log_return_stack`: **[N, L]**
  - `target_bin_stack`: **[N, L]** (>=0 valid, -1 invalid)
  - `target_probs_stack`: **[N, L, B]** (B = number of ordinal bins)
- `target_indices`: **[N_target]** indices selecting "targets" (excludes context tickers)
- `ticker_order`: list[str] length N, consistent ordering across all stacks
- `bin_edges_stack`: list[Tensor] length N, each edges tensor shape **[B+1]**

2.2 Important note: news is effectively disabled in the current cross-ticker dataset

Your `Run101MultiTickerDataset` currently sets:

- `max_k = 0` and pads news to K=0 "to avoid huge dense padding".

That means the emitted tensors are effectively:

- `news_emb_stack`: **[N, L, 0, Dn]**
- `news_mask_stack`: **[N, L, 0]**

So for the **release configuration** where you mostly run price-only, the news path is structurally present but contributes no information (which is consistent with your stated current usage).

If you later want *real* news in the cross-ticker path, you'll need to lift that `max_k=0` constraint and implement masked padding to a reasonable K.

---

3. Model overview (high level)

At a high level, AlphaModel computes:

1) **Price tokens** per ticker per day using a composite encoder:
   - Frozen Kronos-mini encoder (sequence-to-sequence)
   - Trainable numeric MLP branch
   - Token-wise gate combining them
2) Optional **news pooling** conditioned on price tokens (present but K may be 0)
3) **Temporal modeling** per ticker using a causal transformer (TimeLLM)
4) **Cross-sectional modeling** per day via cross-ticker transformer attention
5) **Probabilistic ordinal head** producing return-bin distributions
6) **Expected return** `mu` computed from distribution × bin centers

---

4. Component-by-component (with shapes)

Let: - `N` = number of tickers in the batch - `L` = aligned sequence length - `H` = hidden size (run123 uses **H=320**) - `B` = number of return bins - `K` = number of news slots per day (often 0 in the current cross-ticker dataset)

4.1 Price encoder with MLP gate (Kronos + numeric branch)

Code: - `models/price_encoder_with_mlp.py :: PriceEncoderWithMLP` - `models/price_encoder_with_mlp.py :: build_price_features` - `models/kronos_mini_fullseq.py :: KronosMiniPriceEncoderFullSeq` (frozen)

Per ticker i: - Input: `price_seq_i`: [**1, L, 6**] - Frozen Kronos output: - `h_kronos`: [**1, L, H**] - Numeric engineered features from `build_price_features(price_seq)`: - features: [**1, L, 6**], backward-looking: - `r1` = lr - `r5` = rolling sum over 5 - `r21` = rolling sum over 21 - `vol21` = rolling std over 21 - `hl` = (high-low)/|close| - `volume_z` = normalized volume channel

- Trainable numeric MLP:
  - `h_basic = PriceMLP(features)`: [**1, L, H**]

Token-wise gate and fusion: - `g = sigmoid(GateMLP([h_basic; h_kronos]))`: [**1, L, 1**] - `price_hidden = g * h_basic + (1 - g) * h_kronos`: [**1, L, H**]

Initialization controls: - Gate final layer has: - bias initialized to **-2.0** (strong prior toward Kronos at init) - weights initialized to 0 (gate starts near constant) - Trainer flag `--gate_init_prob` can override bias to `logit(p)` to encourage the MLP path (still learned thereafter).

4.2 News pooling + fusion (structural, often no-op in current release)

Code: - `train/fullseq_fusion.py :: DailyNewsPooler` - `train/fullseq_fusion.py :: GatedPriceNewsFusion`

Per ticker: - Inputs: - `price_hidden`: **[1, L, H]** - `news_emb`: **[1, L, K, Dn]** (K may be 0) - `news_mask`: **[1, L, K]** - `news_intensity`: **[1, L, I]**

- `news_seq = DailyNewsPooler(...)`: **[1, L, H]**
- `fused = GatedPriceNewsFusion(price_hidden, news_seq)`: **[1, L, H]**

When K=0, this stage should be treated as a **present-but-disabled** branch.

4.3 TimeLLM temporal backbone (causal, per ticker)

Code: - `models/time_llm.py :: TimeLLM`

Per ticker: - Input: `fused`: **[1, L, H]** - Masking: - `seq_mask` / `ticker_mask` used as `key_padding_mask` to ignore padded steps - TimeLLM is causal: it cannot attend to future time positions - Output: `out_t = TimeLLM(fused)`: **[1, L, H]**

Memory note: `forward_multi` processes tickers sequentially and uses activation checkpointing around TimeLLM to reduce peak memory.

4.4 Cross-ticker attention (cross-sectional transformer per day)

Code: - `models/cross_ticker_attention.py :: CrossTickerAttention`

After TimeLLM for each ticker: - Collect per-ticker outputs and stack: - `time_out`: **[N, L, H]** - Reshape to cross-sectional view: - `time_out`: **[1, L, N, H]** (B=1 sample; T=L) - `CrossTickerAttention(time_out, ticker_mask)` applies a TransformerEncoder **across tickers** at each time step: - Internally flattens to: **[B*T, N, H]** and uses `src_key_padding_mask` derived from `ticker_mask` - Output remains: **[1, L, N, H]** - If `target_indices` is present, select only targets before the head: - `time_out = time_out[:, :, target_idx, :]`

Finally, the head expects per-ticker sequences, so the tensor is permuted back to: - `head_in`: **[N__target, L, H]**

Key point: This block mixes **cross-section** (tickers) but not time. All temporal causality is handled by TimeLLM.

---

5. Shared ordinal head (distribution over return bins)

Code: - `train/run100_model.py :: SharedOrdinalHead` - `train/run100_model.py :: ResidualAdapter`

Input: `head_in`: **[N__target, L, H]**

Steps: - `adapted = ResidualAdapter(head_in)` → [**N__target, L, H**] - `raw_score = Linear(adapted)` → [**N__target, L**]

Ordinal construction: - Learn `B-1` threshold logits; compute thresholds as: - `thresholds = cumsum(softplus(threshold_logits))` then mean-centered - Learn scalar temperature: - `temp = exp(log_temp)` (clamped)

Compute CDF and probability mass: - `cdf_k = sigmoid((threshold_k - raw_score) / temp)` - Extend CDF with 0 and 1 at ends, then: - `probs = diff(cdf)` → [**N__target, L, B**] - `logits = log(probs)`

Head returns: - `probs`: [**N__target, L, B**] - `logits`: [**N__target, L, B**] - `raw_score`: [**N__target, L**]

---

6. Mu computation (expected return)

Code: - `scripts/run125_alpha_policy.py :: compute_mu`

For each ticker i: - Bin centers: `c = 0.5 * (edges[:-1] + edges[1:])` → [**B**] - Expected return per time step: - `mu[t] = sum_k probs[t,k] * c[k]`

Shape: - `mu`: [**N__target, L**]

`mu` is the canonical scalar alpha signal used downstream (ranking, allocation, etc.).

---

7. Masking, padding, and split behavior

AlphaModel uses masks in three distinct roles:

1) **Padding mask** (`ticker_mask_stack` / `seq_mask`):
   - Prevents padded tokens from contributing to attention or head outputs.
   - In cross-ticker attention, prevents inactive tickers at a given day from participating.
2) **Split mask** (`split_mask_stack[train|val|test]`):
   - Used to gate labels and optionally features.
   - In your `Run101MultiTickerDataset`, you also apply the split mask to features to avoid accidental peeking across split boundaries in non-causal settings.
3) **Target selection** (`target_indices`):
   - Context tickers can be present (e.g., VIX) but excluded from the head and losses.

---

8. Horizon alignment and "shift" conventions (important)

The model **does not shift anything internally**. It outputs distributions at every time step. The mapping from output position `t` to the supervised label is handled by the trainer/evaluator.

This repo supports two equivalent conventions:

Convention A: LM-style teacher forcing (shift in the loss)

Used in `scripts/run100_fullseq_llm.py` when `--full_seq_loss` is enabled (non-cross-ticker path): - Compare `logits[:, :-1]` to `targets[:, 1:]` - Drop the last position

Convention B: Pre-shifted labels (no explicit shift in loss)

In the cross-ticker training branch of `scripts/run100_fullseq_llm.py`, the code currently consumes `target_*` **as-is** (no explicit shift).

Action item for correctness: When training/evaluating the multi-ticker cross-ticker path, confirm that `target_log_return_stack[t]` corresponds to the intended horizon relative to the feature token at position `t`. The safest public write-up statement is:

> "AlphaModel is trained for next-step (t→t+1) forecasting; the one-step alignment is implemented either by shifting in the loss (teacher forcing) or by storing targets pre-shifted in the dataset."

This keeps the write-up truthful and avoids silent off-by-one errors.

---

9. What is frozen vs trainable (release posture)

For the release checkpoint (`outputs/run123/epoch_1.pt`), typical settings are: - Hidden size: **H=320** - TimeLLM: **12 layers**, **4 heads** - Cross-ticker: enabled - Kronos-mini: **frozen** (`requires_grad=False`) - Numeric MLP + gate: trainable - TimeLLM / CrossTickerAttention / Ordinal head: trainable

---

10. Practical notes and known limitations

- **News branch in cross-ticker path is effectively off** today (K=0). The architecture supports it, but the dataset disables it to avoid memory blowups from dense padding.
- The single-ticker path (`Run100FullSeqModel.forward`) supports a **factor bank + factor cross-attention** module. The multi-ticker canonical path (`forward_multi`) currently does not construct a factor bank; it focuses on cross-sectional mixing via CrossTickerAttention.

- The `forward_multi` implementation processes tickers sequentially for memory efficiency; this trades speed for reduced peak VRAM.

---

11. Outputs (runtime dictionary keys)

`Run100FullSeqModel.forward_multi` returns at minimum: - `probs`: [**N__target, L, B**] - `logits`: [**N__target, L, B**] - `raw_score`: [**N__target, L**]

The single-ticker `forward` additionally may expose: - `timing`: operator-level timing breakdown (for profiling) - `price_gate_mean`: average gate value if computed in that path

---

12. Minimal "how to consume AlphaModel" snippet

1) Load a `Run101MultiTickerDataset` batch
2) Run `model(batch)` → get `probs`
3) Convert to `mu` via bin centers
4) Feed `mu` to the portfolio policy model

In this repo, step (3) is implemented in: - `scripts/run125_alpha_policy.py :: compute_mu`